Reddit Tracker — Discovery, Tracking, and Conversational Q&A System for Viral Posts by Everyday Users

Research Proposal

Principal Investigator: Undergraduate Researcher (Reddit username: u/Beautiful-Screen-294) **Institution**: Chung Hua University, Hsinchu, Taiwan **Contact**: b11202076@chu.edu.tw
Project repository: Private (will be opened upon publication of the academic report)

Abstract

Social platforms produce a steady stream of posts by non-influencer ("grassroots") users that suddenly go viral. Readers are drawn in at the moment of discovery and ask "what happened next?" in the comments, but because such posts are not part of any account they already follow, the follow-up is easily missed.

This project builds an end-to-end pipeline that (a) **discovers** candidate viral posts from a curated set of subreddits and keywords, (b) **scores** them with a rule filter plus a small LLM (Claude Haiku) and surfaces a daily Top 5 to a single-digit research audience via Telegram, (c) **tracks** the posts the audience bookmarks by polling four classes of follow-up events (author follow-up posts, author replies, hot replies, crossposts/duplicates), and (d) **answers multi-turn questions** about a tracked event with a heavy-context LLM (Claude Opus) that is fed the full comment tree plus author background and related posts.

The research questions are: **(1)** Does an automated LLM-based scoring pipeline correlate with human bookmarking behavior on everyday-user viral posts? **(2)** Can a heavy-context LLM reliably answer multi-turn questions about a specific user-submitted event, and is the cost justified?

The system is implemented and deployed; it requires read-only public Reddit data through the official OAuth API. Expected request volume is well under the standard 100 QPM rate limit (currently observed at ~9 QPM). This document describes the system design, evaluation plan, cost model, risk analysis, and the ethics framework under which all data is handled.

---

1. Background and Motivation

Across social platforms, an everyday user occasionally posts something that unexpectedly goes viral. Readers want to know how the situation evolves — did the boss back down, did the

relationship recover, did the legal advice work — but they rarely have any subscription relationship to the original poster, so the follow-up slips past.

A prior version of this project (v3) targeted Threads as the observation platform but encountered two structural issues during implementation:

1. **High data-acquisition cost**: Threads public search has required cookie authentication since late 2025, forcing reliance on third-party scrapers (Apify). At the default production scale this cost approximately USD 246/month, with additional risk of actor maintenance interruptions and field-shape drift.
2. **Poor structural support for follow-up tracking**: Threads comments are flat, and quote-style citations are difficult to aggregate. Each follow-up trace required a fresh scraper search.

Reddit is a natural alternative on both points:

- The **official Reddit API is free** within the 100 QPM rate limit, eliminating the scraper layer.
- **Comments are natively threaded**, and three of the four follow-up event classes the project tracks (author follow-up, author reply, hot reply) have direct API endpoints.
- **Subreddits are natural topic containers**, more stable and predictable than open keyword search.
- `submission.duplicates` returns all crossposts and citations in a single call — something v3 could not solve on Threads.

The research questions and the three-stage interaction model (Discover → Push → Bookmark/Track → Q&A) are inherited from v3; only the data layer and per-platform extraction logic differ.

2. Research Objectives

Primary objectives

1. Build an **automated discovery and scoring pipeline** that produces a daily Top 5 candidate list from the target subreddits and keywords.
2. Deliver the daily list via **Telegram bot** at a fixed local time, with inline buttons for three feedback actions (❤️ bookmark, 👎 not interested, 🔕 mute author).
3. Promote bookmarked posts into a **long-term tracking pool** and automatically detect follow-up developments (author's new posts, author's own replies, high-score responses, crosspost citations), with appropriate notification.
4. Provide an `/ask <id>` **conversational Q&A mode** for multi-turn LLM dialogue about a single tracked event.

Secondary objectives

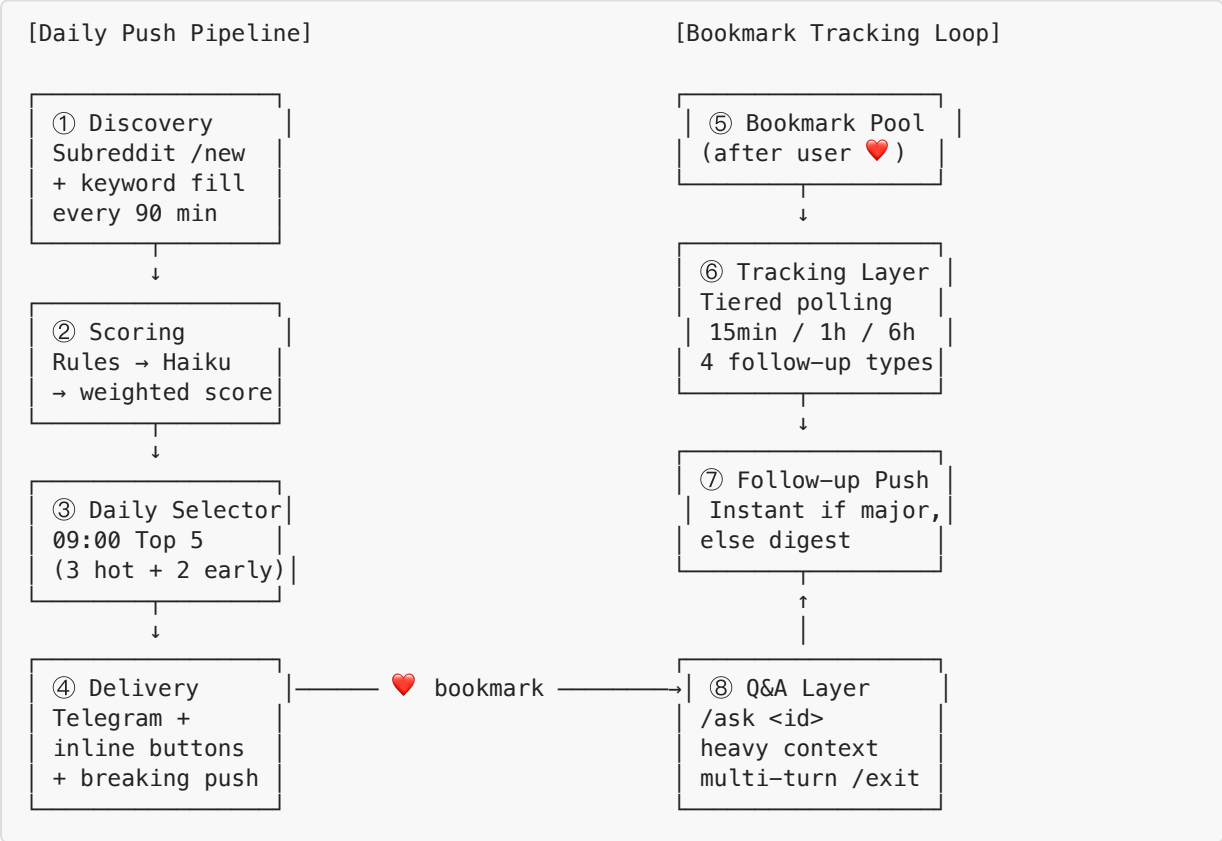
1. Investigate the correlation between algorithmic post-value judgment and human bookmarking behavior.
2. Evaluate Q&A quality of a heavy-context LLM (containing the full comment tree, snapshots, and related posts) on everyday-user events.
3. Produce a reproducible, pseudonymized dataset of grassroots viral posts.

Scope

- **Languages**: Traditional Chinese (r/Taiwan, r/HongKong, r/ChineseLanguage, etc.) and English storytelling subreddits, mixed.
- **Audience**: A single-digit number of opted-in research collaborators who share a single daily push list; no per-user personalization.
- **Platform**: Reddit only. No mixing with Threads, X, Dcard, or other sources.

3. System Architecture

Overall architecture: discovery pipeline + tracking loop



Technology choices

Layer	Technology	Rationale
Backend	Python 3.12 + FastAPI	Lightweight, async-friendly
Database	SQLite (dev) / PostgreSQL (prod-ready)	Single-digit audience; SQLite sufficient
Scheduler	APScheduler	Mature interval/cron triggers
Reddit access	**Official Reddit OAuth API + PRAW 7.x**	Officially supported, predictable rate limit, native comment tree
LLM scoring	Claude Haiku 4.5	Low cost, fast classification
LLM summary + Q&A	Claude Opus / MiniMax	Heavy context, strong reasoning
Notification	python-telegram-bot	Bot framework with inline button support
Deployment	macOS launchd LaunchAgent (local)	Sufficient for the audience size

4. Functional Design

4.1 Discovery layer — subreddit `/new` primary + keyword secondary

Primary strategy: subreddit pool polling. Approximately 12–20 target subreddits are polled every 90 minutes, fetching the most recent 50 posts in each subreddit's `/new` listing. The initial seed list includes:

- **Traditional Chinese subreddits** (4–6): r/Taiwan, r/HongKong, r/translator, r/ChineseLanguage, r/Cantonese, r/China_irl.
- **English storytelling subreddits** (8–14): r/TIFU, r/AmItheAsshole, r/relationship_advice, r/MaliciousCompliance, r/legaladvice, r/UpdateMe, r/offmychest, r/confession, r/taipei, etc.

Subreddits are stored in a `subreddit_sources` table and can be enabled or disabled at runtime; per-source yield and bookmark conversion rate are tracked.

Secondary strategy: keyword search across subreddits. Every 12 hours, a small set of "event-sense" keywords (Chinese: 更新, 後續, 結果, 求助; English: update, found out, caught, exposed, fired, etc.) runs against Reddit search to catch viral posts the primary strategy missed. The keyword pool is intentionally smaller than v3 (20 vs. 30) because subreddits already provide good coverage.

Estimated volume: ~4,800 deduplicated candidates per day across both strategies.

4.2 Scoring layer — three stages

Stage 1: Rule filter. Reddit-adapted hard rules: interaction velocity `(ups + 2·comments) / age_hours > 5`, `author_karma < 10,000`, `account_age > 30 days` (anti-spam), language ∈ {Traditional Chinese, English}, length 100–3,000 characters, not stickied, not distinguished as moderator. Expected pass rate: 8–12%.

Stage 2: Claude Haiku semantic verdict. A five-axis prompt (`story_potential`, `emotional_pull`, `grassroots`, `novelty`, `authenticity`) yields `verdict ∈ {track, skip}`. The grassroots axis is adapted to Reddit's karma signal. Expected pass rate among stage-1 survivors: 30%.

Stage 3: Weighted composite score.

```
final_score = 0.4 · interaction_velocity_normalized
            + 0.3 · semantic_score        # (story + emotional) / 2
            + 0.2 · grassroots_score
            + 0.1 · novelty_score
```

The top N candidates per scoring cycle are inserted into the candidate pool.

4.3 Daily selection — Top 5 = 3 already-hot + 2 early bets

At 09:00 Asia/Taipei (01:00 UTC), the daily push selects:

- **3 already-hot**: top by interaction velocity, posted ≥ 6 hours ago.
- **2 early bets**: posted < 3 hours ago, top by semantic score.

Breaking exception: If a candidate within 1 hour of posting reaches top 1% interaction velocity and `semantic_score > 0.85`, it is pushed immediately. Capped at 2 breaking pushes per day.

4.4 Delivery layer — Telegram

Each push message contains: subreddit, post age, ups, comment count, post title, author handle (karma + account age), event tags, and an 80–120 character LLM summary. Three inline buttons: ❤️ bookmark, 👎 not interested, 🔕 mute author.

Supported commands: `/feed`, `/saved`, `/timeline <id>`, `/ask <id>`, `/exit`, `/digest`, `/settings`.

4.5 Bookmark and tracking layer

After a user bookmarks a post, it is promoted into `tracked_posts` and enters tiered polling:

Stage	Frequency	Items fetched
0–24 h after bookmark	every 15 min	comment tree, author's new posts
1–7 days	every 1 h	full set + crossposts
7–30 days	every 6 h	full set
30 days idle	auto-archived	—

Four follow-up event classes:

Class	Reddit endpoint
`author_followup`	`reddit.redditor(name).submissions.new(limit=20)` (cross-subreddit)
`author_reply`	`submission.comments.list()` filtered by `comment.author == submission.author`
`hot_reply`	`submission.comments.replace_more()` ranked by score (threshold or relative gap)
`crosspost`	`submission.duplicates()` — natively supported, unsolvable on Threads in v3

Detected events are written to `related_posts` and classified by Haiku for "is this a major milestone?". Default delivery is via the daily digest (appended after the day's 5 recommendations); a major milestone (e.g., resolution revealed, original poster appears) is pushed immediately.

4.6 Q&A layer — multi-turn conversation

`/ask <id>` enters Q&A mode for the specified event; subsequent messages are interpreted as questions about that event until `/exit`.

Default context composition (heavy mode):

Block	Content	Estimated tokens
Original post	title + selftext + flair + subreddit	~800
Latest summary	event subject, current state, open questions, expected next steps	~600
Full comment tree	nested replies with indentation	8k–30k
Author background	recent 30 submissions + karma distribution + account age	2k–5k
Follow-up events	all `related_posts` including crossposts	1k–10k

A `/ask <id> --light` flag falls back to original post + summary only, for cost control.

Each user can be in at most one active Q&A session at a time; `/exit` or 5-minute idle ends the session. Conversation history is stored in `qa_sessions` and `qa_messages`.

5. Database Schema

The schema consists of the following primary tables. All tables include sensible indices on `discovered_at`, `posted_at`, foreign keys, and natural unique constraints; full DDL is available in the project repository.

Table	Purpose
`candidate_posts`	All discovered posts, keyed by `reddit_post_id` (Reddit submission id). Stores subreddit, title/selftext, author handle, karma, account creation timestamp, posted-at, discovery source, initial ups/comments, upvote ratio, detected language, and the original raw JSON payload.
`scoring_records`	One row per scoring stage (rules / haiku / final) per candidate, with pass flag, score, structured details, timestamp, and LLM cost.
`daily_pushes`	One row per pushed candidate per push date, with type (`already_hot` / `early_bet` / `breaking`), rank, and push time.
`feedback`	User action on a pushed candidate (`bookmark` / `dislike` / `mute_author`) with timestamp.
`tracked_posts`	Promoted candidates entering long-term tracking, with promotion time, polling tier, last polled time, status, and initial summary.
`post_snapshots`	Time-series snapshots of a tracked post (score, num_comments, upvote_ratio, new-comment summary).
`related_posts`	Detected follow-up events, with relation type (4 classes), relevance score, milestone flag, content, posted-at, discovered-at, and notified-at.
`subreddit_sources` / `keyword_seeds`	Source registries with language hint, enabled flag, last polled time, and yield counters (yielded / promoted / collected).
`qa_sessions` / `qa_messages`	Conversation state for the Q&A layer.
`llm_records`	All LLM call records: provider, model, prompt-token / completion-token counts, cost, latency.

The author handle field stores the original Reddit username; on export for analysis or publication, handles are replaced with a stable salted hash and never released in identifiable form.

6. Evaluation Methodology

6.1 Quantitative metrics

Metric	Definition	Target
Collection Rate	Pushed candidates marked ❤️ by users / total pushed	≥ 30%
Follow-up Hit Rate	Bookmarked events with ≥ 1 detected follow-up within 30 days	≥ 50%

Evaluation window: weeks 4–8 after stable system operation.

6.2 Qualitative analysis

3–5 case studies tracing the full lifecycle: discovery basis, bookmark timing, follow-up arrival, Q&A quality. At least one case is required to involve a crosspost-mediated follow-up (the v4-vs-v3 differentiator).

6.3 Q&A quality evaluation

30 (question, answer) pairs sampled from collected Q&A sessions. The researcher self-rates on three dimensions: factual correctness (0–1), citation precision (0–1), and presence of hallucination (binary).

6.4 Reddit-specific comparative measurements

- Distribution of the four follow-up event classes (crosspost share, expected high under Reddit's native support).
- Collection-rate comparison between Chinese-language and English-language subreddits (cultural / linguistic factor).

7. Project Milestones

Milestone	Theme	Status
M1	Reddit API feasibility verification	✓ done
M2	Discovery layer + schema	✓ done
M3	Scoring layer connected to Reddit	✓ done
M4	Daily selection + push layer	✓ done
M5	Bookmark tracking layer (4 follow-up types)	✓ done
M6	Q&A layer (multi-turn, heavy context)	✓ done
M7	**Evaluation data collection + tuning**	**in progress — blocked on data access**
M8	Final report and demo	pending M7

M1–M6 are implemented, tested, and deployed via launchd on local infrastructure. Production data collection began on 2026-05-25. The current blocker is Reddit's deprecation of anonymous `.json` public endpoint access, observed starting 2026-05-29; this proposal requests OAuth-based access to complete M7 and M8.

Expected completion: M7 evaluation and M8 academic report by end of 2026-Q3.

8. Cost Estimation

Item	Daily	Monthly
Reddit API (discovery + tracking)	$0 (within free tier)	$0
Claude Haiku scoring (~200 calls)	$0.20	$6
Claude Opus summary (5 posts)	$0.25	$7.50
Claude Opus Q&A (5 sessions with cache)	$0.40–0.75	$12–22
Total estimated	**~$1.20**	**$15–36**

Prompt caching on the heavy-context Q&A path (same `tracked_post_id` reused within 5 minutes) reduces effective Q&A cost by approximately 50%. A daily hard cap of USD 5 on LLM spend serves as a fuse.

9. Risk Analysis

Risk	Mitigation
Reddit API policy change	This proposal is the response to the most recent change. Access requested for academic, non-commercial use only; usage maintained well below 60 QPM (40% buffer below the 100 QPM limit). Fallback: user-submitted URL mode for single-post tracking.
Low Chinese-language sample volume	English subreddits provide primary sample; Chinese subreddits as secondary comparison. Report explicitly states language distribution; no overgeneralization.
Low collection rate (< 10%)	Use feedback to re-tune Haiku prompt; increase early-bet quota to test taste for novelty.
Q&A token cost overrun	Prompt cache enabled by default; auto-downgrade to medium context (top 30 comments) when daily cost approaches cap; soft cap on per-day Q&A count.
Compliance and privacy	See §10 Ethics and Data Protection.

10. Ethics and Data Protection

This is the section most directly relevant to the Reddit reviewer; it is enumerated in full.

10.1 Lawful basis and use

- The project is conducted as **non-commercial academic research** at Chung Hua University.
- All collected data is **publicly available** through the Reddit web interface. No private subreddits, modmail, deleted content, or content visible only to logged-in users is accessed.
- The system **complies with Reddit's User Agreement, Privacy Policy, and the Responsible Builder Policy**. The researcher has reviewed all three documents prior to this submission.

10.2 Audience and exposure

- The Telegram delivery audience is a **single-digit number of opted-in research collaborators** (the principal investigator plus a small number of project-affiliated readers who have explicitly agreed to participate).
- The system **does not republish Reddit content externally**, does not power a public API, and does not feed any third-party application.

- The dataset is not used to train, fine-tune, or evaluate generative models. LLM calls are read-only over the data and produce only ephemeral inference outputs.

10.3 Pseudonymization and identifiers

- Reddit usernames are **stored in their original form for tracking continuity** during the active research period, then **replaced with a stable salted hash** before any dataset export, case study, or report publication.
- No real-name, email, IP, or other PII beyond what already appears on a Reddit user's public profile is collected or inferred.
- Case studies in the final report **use pseudonyms** and **redact or paraphrase identifying details** (specific employers, locations, unique life events) sufficiently to prevent re-identification.

10.4 Retention and deletion

- Active retention is **limited to the study duration** (estimated 6 months from study start to final report publication).
- A `/forget <id>` Telegram command **completely deletes** all records associated with a given tracked event from the database (candidate, scoring records, snapshots, related posts, Q&A history).
- After the study concludes, the operational database is archived in pseudonymized form for reproducibility; the active live database is wiped.
- A user who requests removal of their content from the dataset (via any channel) has their content deleted within 7 days.

10.5 Rate limiting and platform respect

- Read requests are scheduled with conservative pacing: subreddit listings polled every 90 minutes, keyword search every 12 hours, bookmarked-post deep polling tiered from 15 minutes (first day) down to 6 hours (after one week).
- Observed steady-state request rate is approximately 9 QPM, well below the 100 QPM Reddit OAuth ceiling.
- All requests carry an honest User-Agent identifying the script, the version, and the Reddit username of the operator, per Reddit's API guidelines.

10.6 Report and publication

- The final academic report includes a dedicated ethics chapter discussing social media research ethics, anonymization rationale, and limitations.
- Any case study published in the report or accompanying materials is reviewed for re-identification risk before release.

11. Expected Deliverables

1. A fully working Telegram bot demonstration system.
2. A public GitHub repository (opened after report publication).
3. An academic project report (Traditional Chinese, approximately 30 pages).
4. A 5-minute demo video.
5. 3–5 case studies tracing the full lifecycle of selected events from push to bookmark to follow-up.
6. Quantitative evaluation data from at least 4 weeks of stable operation (collection rate, follow-up hit rate, Chinese-vs-English comparison).
7. Self-evaluated quality scores on 30 (question, answer) Q&A pairs.

12. References

- Reddit API documentation: https://www.reddit.com/dev/api/
- Reddit Responsible Builder Policy
- PRAW (Python Reddit API Wrapper): https://praw.readthedocs.io
- python-telegram-bot: https://docs.python-telegram-bot.org
- Anthropic Claude API: https://docs.claude.com
- APScheduler: https://apscheduler.readthedocs.io
- FastAPI: https://fastapi.tiangolo.com